Research Note: LLD Practice Platform

1. The Learner Problem

Low-Level Design (LLD) / Object-Oriented Design (OOD) is notoriously easy to attempt but difficult to self-evaluate.

When candidates practice designing systems like a **Parking Lot**, **Elevator System**, **Vending Machine**, or **Food Delivery App**, they can quickly sketch out classes, interfaces, and methods. However, they are left with fundamental uncertainties:

- *Are these class responsibilities cohesive, or did I create a god-object?*
- *Is this abstraction actually decoupling components, or is it premature complexity?*
- *How fragile is this design when requirements inevitably change (e.g., adding EV charging spots or dynamic surge pricing)?*
- *Are edge cases, race conditions, and thread safety adequately addressed?*

In traditional algorithmic coding (e.g. LeetCode), automated test runners provide immediate deterministic feedback (Pass/Fail, Time/Space Complexity). In LLD, **there is rarely a single "correct" solution**; valid designs differ based on assumptions, trade-offs, and extensibility goals. Without structured evaluation, learners either assume their design is fine or get lost in subjective debates.

---

2. Current Practice Methods & Market Analysis

Platform / Approach	Primary Focus	Strengths	Critical Gaps
LLDCanvas	Visual UML editing, Draft notation, Code execution, timed practice	Interactive diagramming surface; class connection visualization; in-browser code editor.	Focuses primarily on visual drawing and boilerplate coding rather than structured, rubric-based feedback on architectural reasoning and trade-offs.
InstaMock	AI mock interviews for system design & LLD	Real-time conversational interview simulation; audio/chat interaction mimics recruiter pressure.	Feedback is delivered as broad conversational summaries rather than structured, criterion-by-criterion evidence grounded in specific design decisions.
NeetCode OOD	Interactive coding & Object-	Structured curriculum; clear explanations of	Primarily video and boilerplate code walkthroughs; lacks automated, rubric-grounded

Platform / Approach	Primary Focus	Strengths	Critical Gaps
	Oriented Design video tutorials	design patterns and object modeling.	assessment of candidate design reasoning.
Refactoring.Guru	Design patterns and SOLID principles reference	Visual explanations of creational, structural, and behavioral patterns; clear intent and trade-offs.	Purely static reference documentation; provides no submission, evaluation, or deliberate practice loop.
Traditional Sources (Educative Grokking LLD, Awesome LLD)	Static tutorials, reference repos, and interview guides	Comprehensive walkthroughs of canonical designs (Parking Lot, Movie Booking, Elevator).	Completely passive learning. Learners inspect finished solutions without validating their own design reasoning or learning through iterative revision.
Generic LLMs (ChatGPT / Claude)	Free-form prompt querying ("review my LLD code")	Instant feedback; versatile conversational interaction.	Unconstrained scoring; arbitrary ratings without grounded rubrics; hallucinated critique not anchored to candidate quotes; score drift across attempts.

3. Observed Facts vs. Product Assumptions

To ground our product strategy in rigorous engineering evidence, we explicitly separate observed empirical facts from design hypotheses:

Observed Facts (Empirical Market Data)

1. **Rubric Absence Breeds Inconsistency**: Generic LLM feedback swings wildly depending on phrasing. Without a fixed 100-point rubric, the same submission scored on two separate calls can fluctuate by 20+ points.
2. **Hallucination Risk on Omissions**: When a candidate omits a requirement (e.g., concurrency in a Parking Lot), generic LLMs frequently hallucinate that the candidate addressed it or criticize imaginary code never written.
3. **Diagramming Tooling Friction**: Visual canvas tools frequently shift learner cognitive effort toward aligning graphical boxes and connecting relationship arrows rather than articulating class responsibilities, interfaces, and design pattern trade-offs.
4. **Disjointed Revision**: Existing tools do not store chronological, immutable attempts linked to structured evaluations. Learners cannot see whether their score increased or decreased following a refactoring.

Product Assumptions (Hypotheses Under Test)

1. **Text-First Modeling Accelerates Feedback**: Describing classes, methods, relations, and trade-offs in structured markdown allows learners to articulate architecture faster than visual diagramming while being directly evaluatable by LLM engines.
2. **Evidence-Grounded Critique Prevents Hallucination**: Requiring every evaluation criterion to either cite a verbatim quotation from the candidate's submission or explicitly declare that the component is missing eliminates false praise and hallucinated critique.
3. **Deterministic Score Aggregation Builds Trust**: Computing total scores on the server by summing validated criterion points—rather than letting an LLM hallucinate an arbitrary total score—provides consistent, trustworthy evaluation.
4. **Direct Attempt Comparison Drives Mastery**: Displaying score trajectory badges (e.g., `+15 pts`, `−5 pts`, `Baseline`) in attempt history gives learners tangible progress signals that motivate iterative refinement.

---

4. Why This MVP Is Different

Unlike visual canvas editors, conversational chatbots, or passive video tutorials, this platform is purpose-built as a **deliberate practice loop for low-level design**:

1. **Standardized 100-Point Rubric**: Every evaluation grades exactly the seven standardized dimensions defined in the database:
    - **Requirement Understanding** (15 pts): Identifies core functional scope, boundaries, and assumptions without over-engineering.
    - **Class Responsibilities** (20 pts): Enforces Single Responsibility Principle (SRP) and avoids god-objects.
    - **Encapsulation & Interfaces** (15 pts): Exposes clear, minimal public contracts and protects internal state.
    - **Coupling & Cohesion** (15 pts): Loose coupling between subsystems and high internal cohesion.
    - **Abstraction / Design Patterns** (10 pts): Deliberate, appropriate design pattern selection (Strategy, Factory, State, Observer).
    - **Extensibility** (15 pts): Open-Closed Principle (OCP); handles new types/rules without rewriting core logic.
    - **Edge Cases & Testability** (10 pts): Concurrency safety, boundary conditions, and isolation for testing.
2. **Evidence Grounding Engine**: Every feedback card includes verbatim candidate evidence, identified architectural concerns, actionable next steps, and evaluator confidence.

3. **Save-Before-Evaluate Reliability**: The candidate's submission is permanently committed to PostgreSQL *before* evaluation starts. Even if an upstream LLM times out or hits rate limits, student work is never lost and evaluation can be retried in-place.
4. **Measurable Improvement Tracking**: Attempts are immutable records. Learners can review past attempts, see score differences, and systematically fix identified architectural weaknesses.

---

5. The Practice Loop